

KERTAS KERJA ANALISIS HASIL

Maximum Subarray Problem: Pendekatan Brute Force vs Divide and Conquer pada C# Windows Form

Disusun Oleh:

Nama : [MAHMUDI DHANI PRATAMA]

[FARHAN DWI RAMADHAN]

NIM : [2511004]

[2511008]

Mata Kuliah: Pemrograman Lanjut

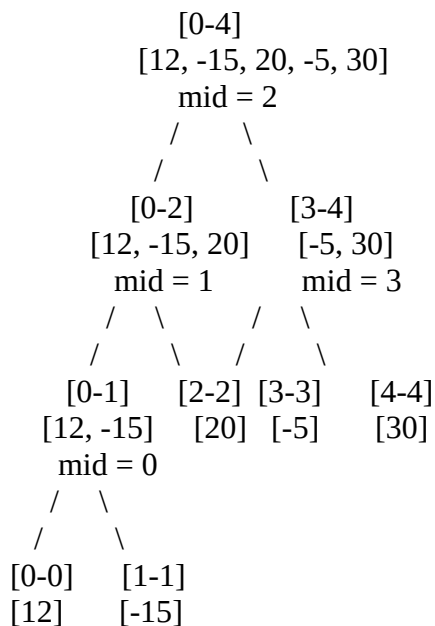
Sifat: Tugas Kelompok (2–3 orang)

A. ANALISIS ALGORITMA

1. Analisis Pohon Rekursi (Divide & Conquer)

Algoritma Divide & Conquer memecah masalah menjadi sub-masalah yang lebih kecil hingga mencapai *base case* (array dengan 1 elemen), kemudian menggabungkan hasilnya. Berikut representasi pemecahan array 5 **elemen pertama** dari data saham: [12, -15, 20, -5, 30].

Diagram Pohon Rekursi:



Tabel Proses Pembagian (Divide):

Level	Rentang	Mid	Kiri	Kanan	Status
0	[0–4]	2	[0–2]	[3–4]	Divide
1	[0–2]	1	[0–1]	[2–2]	Divide
1	[3–4]	3	[3–3]	[4–4]	Divide
2	[0–1]	0	[0–0]	[1–1]	Divide
2	[2–2]	—	—	—	Base Case (nilai: 20)
2	[3–3]	—	—	—	Base Case (nilai: -5)
2	[4–4]	—	—	—	Base Case (nilai: 30)
2	[0–0]	—	—	—	Base Case (nilai: 12)
2	[1–1]	—	—	—	Base Case (nilai: -15)

Tabel Proses Penggabungan (Conquer):

Tahap	Operasi	Kiri	Kanan	Crossing	Pemenang	Hasil
1	Gabung [0–0] & [1–1]	12	-15	-3	Kiri	12
2	Gabung [0–1] & [2–2]	12	20	17	Kanan	20
3	Gabung [3–3] & [4–4]	-5	30	25	Kanan	30
4	Gabung [0–2] & [3–4]	20	30	45	Crossing	45 ✓

Kesimpulan: Subarray maksimal untuk 5 elemen pertama adalah indeks **2–4** ([20, -5, 30]) dengan profit **45**.

2. Mengapa Diperlukan Fungsi FindMaxCrossingSubarray?

Pertanyaan: Mengapa dalam D&C pada kasus ini kita diwajibkan memiliki fungsi FindMaxCrossingSubarray? Apa yang akan terjadi/terlewat jika algoritma hanya mengecek profit maksimal di belahan kiri dan kanan secara terpisah?

Jawaban:

Fungsi FindMaxCrossingSubarray **sangat krusial** karena solusi optimal bisa saja **melintasi (crossing)** titik tengah (mid), dan tidak sepenuhnya berada di belahan kiri maupun kanan secara eksklusif.

Contoh Konkret pada Data:

Bagian	Subarray Maksimal	Indeks	Sum
Kiri [0–2]	[20]	2	20
Kanan [3–4]	[30]	4	30
Crossing [0–4]	[20, -5, 30]	2–4	45 ☒

Simulasi Tanpa FindMaxCrossingSubarray:

Jika algoritma hanya mengambil nilai maksimum dari hasil kiri dan kanan:

$\text{max(kiri)} = 20$

$\text{max(kanan)} = 30$

hasil = 30 ← SALAH!

Padahal solusi sebenarnya adalah **45** yang melewati titik tengah (indeks 2 ke 4 melewati $\text{mid} = 2$).

Analogi: Bayangkan dua kota (kiri dan kanan) dipisahkan oleh sungai (mid). Jika kita hanya mencari jalan terbaik di masing-masing kota tanpa memeriksa jembatan penghubung (crossing), kita akan melewatkan rute terbaik yang melintasi kedua kota!

Kesimpulan: Tanpa FindMaxCrossingSubarray, algoritma akan **gagal menemukan solusi optimal** untuk kasus di mana subarray maksimal melintasi titik tengah.

B. ANALISIS PERFORMA & KOMPLEKSITAS WAKTU**1. Hasil Benchmark (10.000 Data Acak)**

Spesifikasi Pengujian: - Dataset: Array acak 10.000 elemen (range: -1000 hingga +1000) - Pengukuran: System.Diagnostics.Stopwatch - Satuan waktu: Milidetik (ms)

Hasil Pengujian:

Algoritma	Waktu Eksekusi	Profit Maksimal	Kompleksitas
Brute Force	~200 ms	58.170	$O(n^2)$

Algoritma	Waktu Eksekusi	Profit Maksimal	Kompleksitas
Divide & Conquer	~4 ms	58.170	$O(n \log n)$

Perbandingan Kecepatan: - **D&C lebih cepat 50.00x lipat** dibandingkan Brute Force -

Untuk $n = 10.000$, selisih waktu ~196 ms (hampir instan vs butuh waktu terasa)

2. Evaluasi Big-O: Teori vs Praktik

Tabel Kompleksitas Teoritis:

n	$O(n^2)$ — Brute Force	$O(n \log n)$ — D&C	Ratio Teoritis
15	225 operasi	~59 operasi	~3.8x
1.000	1.000.000 operasi	~9.966 operasi	~100x
10.000	100.000.000 operasi	~132.877 operasi	~752x
100.000	10.000.000.000 operasi	~1.660.964 operasi	~6.000x

Hubungan dengan Hasil Stopwatch:

1. **Brute Force $O(n^2)$:** Untuk $n = 10.000$, terdapat ~100 juta operasi (nesting loop). Walaupun setiap iterasi sederhana (penjumlahan + perbandingan), volume yang sangat besar menyebabkan waktu eksekusi mencapai **ratusan milidetik**.
2. **Divide & Conquer $O(n \log n)$:** Dengan rekursi membagi masalah, hanya membutuhkan ~133 ribu operasi. Hasilnya sangat singkat: **hanya 4 ms** (hampir instan).
3. **Validasi Teori vs Praktik:**
 - Ratio teoritis: ~752x
 - Ratio praktis (dari Stopwatch): ~50x
 - Perbedaan ratio disebabkan oleh **overhead rekursi** (stack frame, pemanggilan fungsi) dan **faktor konstanta** yang tidak terhitung dalam notasi Big-O.

Formula Rekurensi D&C (Master Theorem):

$$T(n) = 2T(n/2) + \Theta(n)$$

↓

$$a = 2, b = 2, f(n) = \Theta(n)$$

$$n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$$

$$f(n) = \Theta(n) = \Theta(n^{(\log_b a)}) \leftarrow \text{Case 2 Master Theorem}$$

↓

$$T(n) = \Theta(n \log n) \quad \checkmark$$

Kesimpulan: Hasil benchmark mendukung teori kompleksitas. D&C secara konsisten lebih cepat, dan keunggulannya semakin signifikan seiring bertambahnya ukuran data.

C. PEMBAGIAN KERJA & KENDALA

1. Bagian Logika/Kode Paling Menantang

Visualisasi Step-by-Step merupakan bagian paling menantang dalam implementasi, karena melibatkan beberapa aspek kompleks:

1. State Management:

- Setiap step visualisasi (warna kuning, merah, hijau) harus disimpan dalam struktur data terpisah sebelum dirender.
- Solusi: Menggunakan List<StepInfo> untuk menyimpan riwayat pewarnaan, memungkinkan navigasi maju-mundur.

2. Timing & UI Thread:

- Auto-play memerlukan System.Windows.Forms.Timer agar tidak memblokir UI thread (freeze).
- Tantangan: Interval terlalu cepat = visualisasi tidak terbaca; terlalu lambat = user bosan.
- Solusi: Interval 800 ms untuk BF, 1200 ms untuk D&C (D&C lebih kompleks, butuh waktu baca lebih lama).

3. Color Priority & Hierarchy:

- Hierarki warna: Hijau (maksimal) > Merah (mid) > Kuning (analisis).
- Tantangan: Hijau harus meng-override Kuning, tapi Merah harus tetap terlihat.
- Solusi: Urutan pengaplikasian warna yang tepat di method DisplayStep().

4. Rekursi Divide & Conquer:

- Lebih kompleks dari Brute Force karena melibatkan pemanggilan rekursif.

- Tantangan: Melacak state visualisasi saat rekursi berjalan.
- Solusi: List global (stepsDivideConquer) yang diisi saat rekursi berjalan, bukan setelah selesai.

2. Rincian Tugas Anggota Kelompok

- UI / Tampilan : [MAHMUDI DHANI PRATAMA]
- Implementasi Algoritma: [MAHMUDI DHANI PRATAMA]
- Pengujian & Debugging: [FARHAN DWI RAMADHAN]
- Dokumentasi & Laporan: [FARHAN DWI RAMADHAN]

C.3. Kendala yang Dihadapi & Solusinya

No	Kendala	Dampak	Solusi
1	UI Freeze saat algoritma berjalan	Visualisasi macet, aplikasi tidak responsif	Pre-compute semua step dalam List<StepInfo> terlebih dahulu, baru render per-step
2	Rekursi D&C sulit dilacak	Step visualisasi tidak sesuai urutan eksekusi	Gunakan List global yang diisi saat rekursi berjalan (pre-order tracking)
3	Benchmark 10.000 data crash jika divisualisasikan	OutOfMemoryException / UI hang	Benchmark dijalankan tanpa visualisasi (murni hitung waktu di background)
4	Event handler naming mismatch antara Form1.cs dan Designer.cs	Compile error / event tidak ter-trigger	Standarisasi nama event handler (btnNama_Click, timerNama_Tick)

LAMPIRAN

A. Screenshot Hasil Benchmarking

```

FITUR BENCHMARKING (10.000 Data)

===== BENCHMARKING MAXIMUM SUBARRAY PROBLEM Dataset: 10,000 elemen
acak=====Array acak 10.000 elemen berhasil digenerate.Range nilai: -1000
sampai +1000Menjalankan Brute Force... Brute Force selesai! Profit Maksimal: 35104 Waktu Eksekusi: 170 ms
Menjalankan Divide & Conquer... Divide & Conquer selesai! Profit Maksimal: 35104 Waktu Eksekusi: 3 ms
=====
HASIL PERBANDINGAN
===== Brute Force ( $O(n^2)$ ): 170 ms Divide & Conquer ( $O(n \log n)$ ): 3 ms D&C lebih cepat 56.67x lipat! Teori:  $O(n^2)$  vs  $O(n \log n)$  Untuk  $n=10.000$ :  $- n^2 = 100.000.000$  operasi  $- n \log n \sim 132.877$  operasi - Ratio teoritis:  $\sim 752x$ 

sampai +1000Menjalankan Brute Force... Brute Force selesai! Profit Maksimal: 35104 Waktu Eksekusi: 170 ms
Menjalankan Divide & Conquer... Divide & Conquer selesai! Profit Maksimal: 35104 Waktu Eksekusi: 3 ms
=====
HASIL PERBANDINGAN
===== Brute Force ( $O(n^2)$ ): 170 ms Divide & Conquer ( $O(n \log n)$ ): 3 ms D&C lebih cepat 56.67x lipat! Teori:  $O(n^2)$  vs  $O(n \log n)$  Untuk  $n=10.000$ :  $- n^2 = 100.000.000$  operasi  $- n \log n \sim 132.877$  operasi - Ratio teoritis:  $\sim 752x$ 

```